

Анализ графов

Пентегова Мария Сергеевна

Группа: 25.Б41-мм

Кафедра: системного программирования

Научный руководитель: Григорьев Семён Вячеславович

Номер семестра практики: 2

1. Введение

GraphBLAS позволяет формулировать алгоритмы анализа графов в терминах операций линейной алгебры над разреженными матрицами и векторами. В данной работе исследуется обход графа в ширину (BFS) и сравниваются классическая реализация на основе CSR с реализацией на SuiteSparse:GraphBLAS.

Цель работы — реализовать BFS в нескольких вариантах и исследовать влияние структуры графа на производительность классического и матричного подходов.

2. Постановка задачи и реализация

Реализованы четыре варианта алгоритма: классический BFS с построением массива родителей (parent-BFS), классический multisource BFS, GraphBLAS BFS с вычислением уровней и GraphBLAS multisource BFS.

Для классической реализации граф хранится в формате CSR (Compressed Sparse Row). Для вершины v её соседи находятся в диапазоне

$$[\text{row_ptr}[v], \text{row_ptr}[v + 1])$$

массива `col_idx`. Обычный BFS использует очередь и при первом посещении вершины u записывает `parent[u] = v`. Для multisource BFS все исходные вершины добавляются в очередь до начала обхода. Классический BFS имеет асимптотическую сложность $O(|V| + |E|)$.

В GraphBLAS граф представляется булевой матрицей смежности A , а текущий фронт BFS — разреженным булевым вектором f_k . Следующий фронт вычисляется посредством векторно-матричного умножения

$$f_{k+1} = f_k A,$$

при этом используются булевы операции OR и AND. В реализации этому соответствует `semiring LOR - LAND`. Уже посещённые вершины исключаются при помощи дополненной маски `visited`, после чего новый фронт добавляется к множеству посещённых вершин. Номер итерации записывается как уровень BFS.

Таким образом, GraphBLAS-реализация непосредственно выражает шаг BFS через линейно-алгебраическую операцию над разреженной матрицей. Для multisource BFS начальный фронт содержит несколько исходных вершин одновременно, поэтому дальнейшие уровни вычисляются одной последовательностью матрично-векторных операций.

Программа написана на C11. Используются CMake, SuiteSparse:GraphBLAS, CTest и POSIX-функция `clock_gettime()` с `CLOCK_MONOTONIC`. Матрица GraphBLAS импортируется из подготовленного CSR-представления.

3. Эксперимент

Для экспериментов выбраны семь графов из SuiteSparse Matrix Collection. Для однократного BFS в качестве источника выбиралась вершина с максимальной степенью. Для multisource BFS ис-

пользовались четыре равномерно распределённых источника. Перед измерениями выполнялся дополнительный прогревочный запуск, после чего каждая реализация запускалась 10 раз.

Граф	$ V $	$ E $	Тип	Степень источника
com-Orkut	3.07M	117M	социальный	33313
soc-LiveJournal1	4.85M	69M	социальный	38660
road_usa	23.95M	28.85M	дорожный	4
roadNet-CA	1.97M	2.77M	дорожный	3
web-Google	0.92M	5.10M	веб-граф	6332
cit-Patents	3.77M	16.52M	цитирование	852
com-Amazon	0.33M	0.93M	социальный	546

Таблица 1: Характеристики использованных графов.

Время измерялось в микросекундах. В таблице приведены средние значения по 10 запускам; минимальные и максимальные значения использовались для контроля разброса результатов.

Граф	CP	CM	GB-L	GB-M	CP/GB-L
com-Orkut	2652628	2700768	486500	420307	5.45x
soc-LiveJournal1	1915627	1976468	358255	331458	5.35x
road_usa	1451387	1588466	25839879	22121848	0.06x
roadNet-CA	127543	110558	663630	670218	0.19x
web-Google	100233	80683	101149	106972	0.99x
cit-Patents	17500	4730	181639	46757	0.10x
com-Amazon	38967	31974	61923	62144	0.63x

Таблица 2: Среднее время BFS по 10 запускам, мкс. CP — Classic Parent, CM — Classic Multisource, GB-L — GraphBLAS Level, GB-M — GraphBLAS Multisource.

4. Анализ результатов

На **com-Orkut** и **soc-LiveJournal1** GraphBLAS-реализация оказалась существенно быстрее классической: для однократного BFS получено 486 500 и 358 255 мкс против 2 652 628 и 1 915 627 мкс. Ускорение составляет соответственно 5.45 и 5.35 раза.

На дорожных графах наблюдается противоположный результат. Для **road_usa** классический BFS занимает 1 451 387 мкс, тогда как GraphBLAS Level — 25 839 879 мкс. Для **roadNet-CA** получено 127 543 и 663 630 мкс соответственно. На **web-Google** времена близки: 100 233 и 101 149 мкс.

На **cit-Patents** классическая реализация также быстрее: 17 500 против 181 639 мкс. Для **com-Amazon** различие меньше: 38 967 против 61 923 мкс.

Полученные результаты показывают, что использование GraphBLAS не даёт одинакового выигрыша на всех графах. На двух социальных графах из исследованной выборки матричная реализация значительно быстрее, тогда как на четырёх графах классическая реализация имеет преимущество, а на **web-Google** результаты практически совпадают.

Одним из возможных объяснений является различие структуры фронтов BFS. Широкий фронт содержит большое количество вершин, которые могут быть обработаны одной матричной операцией, тогда как при узком фронте последовательный обход CSR может иметь меньшие накладные расходы. Однако ширина фронта в данной работе непосредственно не измерялась, поэтому этот фактор следует рассматривать как гипотезу, которую можно проверить в дальнейшем. Полученные данные также показывают, что ориентированность графа сама по себе не определяет производительность: например, **cit-Patents** является ориентированным графом, но классическая реализация существенно быстрее.

5. Тестирование и заключение

Корректность реализации проверяется с помощью CTest и `assert.h`. Тесты выполняются на небольшом тестовом графе и проверяют посещение вершин, вычисление расстояний, построение родительского дерева, работу multisource BFS, а также соответствие результатов классической и GraphBLAS-реализаций. Все тесты проходят успешно (100% passed).

В работе реализованы четыре варианта BFS: классический BFS с построением дерева обхода, классический multisource BFS, GraphBLAS BFS с вычислением уровней и GraphBLAS multisource BFS. Классическая реализация использует очередь и CSR-представление графа, а GraphBLAS реализация формулирует переход между уровнями BFS как операции над разреженной матрицей смежности и вектором текущего фронта.

Эксперименты проведены на семи графах из SuiteSparse Matrix Collection. Полученные результаты показывают существенную зависимость времени выполнения от структуры графа. На `com-Orkut` и `soc-LiveJournal1` GraphBLAS BFS оказался быстрее классического варианта примерно в 5.45 и 5.35 раза соответственно. На `road_usa`, `roadNet-CA`, `cit-Patents` и `com-Amazon` классическая реализация оказалась быстрее. На `web-Google` времена двух подходов были близки.

Таким образом, по результатам эксперимента нельзя выделить один универсально более быстрый способ реализации BFS. GraphBLAS оказывается конкурентоспособным на графах, где обработка большого фронта BFS позволяет эффективно использовать операции над разреженными матрицами. Классический BFS оказывается предпочтительнее по времени на графах с небольшими фронтами, где накладные расходы матричных операций GraphBLAS не компенсируются преимуществами выбранного представления.

При этом результаты не позволяют сформулировать универсальное правило только по типу графа, его ориентированности или числу рёбер. Например, ориентированный граф `cit-Patents` показал преимущество классического BFS, тогда как ориентированные социальные графы `com-Orkut` и `soc-LiveJournal1` показали преимущество GraphBLAS. Следовательно, для более точного объяснения различий необходимо учитывать непосредственно структуру BFS-фронтон.

В рамках данной работы практический выбор можно сформулировать следующим образом: классический BFS целесообразно использовать как базовый вариант с небольшими накладными расходами, особенно при узких фронтах, тогда как GraphBLAS представляет интерес для графов, на которых уровни BFS содержат большое количество одновременно обрабатываемых вершин. Это является наблюдением для исследованного набора графов, а не универсальным правилом для всех графов.

В качестве дальнейшего эксперимента полезно измерять число вершин в каждом BFS-фронте и сравнить его с временем выполнения каждой реализации. Это позволит непосредственно проверить связь между шириной фронта и преимуществами матричной реализации.

Репозиторий: https://github.com/MariaPentegova/graph_blas_bfs-2

6. Список использованных источников

1. IEEE Std 1003.1-2017. *POSIX.1-2017: System Interfaces*. The Open Group.
2. GHOST966. *Алгоритмы на графах: BFS — поиск в ширину*. Хабр, 2025.
3. Intel Corporation. *Intel oneMKL: Sparse Storage Formats*. 2025.
4. GraphBLAS Forum. *The GraphBLAS C API Specification*. Version 2.1, 2023.
5. Davis T. A. *SuiteSparse:GraphBLAS*. ACM Transactions on Mathematical Software, 2023.
6. *SuiteSparse Matrix Collection*.